7SENG014W Web Application Development 2026

Coursework 01: Backend API (Dot Net Core)

Coursework Description:

Overview:
This coursework is an individual project where you are required to design and develop a fully functional .NET Core Backend API (You can decide the context of your application yourself). The project will involve a wide range of skills, including project architecture, database design, API development, authentication, business logic implementation, unit testing, and deployment. You will need to demonstrate your understanding of both theoretical concepts and practical implementation.

Key Features:

- The API must adhere to RESTful principles.
- You are required to implement a solid backend architecture, including the use of controllers, models, services, and a repository pattern.
- Authentication and authorization must be handled securely, using industry-standard methods like JWT tokens and ASP.NET Core Identity.
- You will need to use Entity Framework Core for database interaction, implement migrations, and manage relationships between entities.
- Unit testing is required for all critical components, with a focus on testing API endpoints and services.

Important Requirements:

1. **Individual Work:** This coursework must be completed independently. You should not collaborate with others or share code. Any form of plagiarism will not be tolerated, and appropriate academic penalties will be applied, including potential failure of the coursework.
2. **Viva:** After submitting your project, you will need to attend a viva (oral exam) where you will explain your design decisions and implementation details. The viva is critical; failure to explain key aspects of your project may result in a deduction of marks. It is your responsibility to ensure that you can articulate your design choices and technical implementation.
3. **Model Requirements:** Your API must include at least five models (entities) to ensure the project meets the complexity requirements. These models should interact with each other through relationships (e.g., one-to-many, many-to-many).
4. **Ethical Conduct:** Please remember that academic integrity is vital. Any attempts to deceive, including plagiarizing code or not properly crediting external sources, will be subject to disciplinary actions. Ensure that your work is authentic and demonstrates your personal effort and understanding.

Submission Guidelines:

You are required to submit the following for your coursework:

1. **GitHub Repository Link:** Submit the link to your GitHub repository where all the code for your project is stored. Ensure that your repository is properly organized and follows the required folder structure.
2. **Code Files & Dockerfile:** You must include all source code, including your **Dockerfile**, in a **zipped** version of your project. The zipped file should contain everything required to run the project, including but not limited to code files, configuration files, and any other necessary resources.
3. **Submission Deadline:** All submissions must be uploaded to **Blackboard** no later than **02nd March 2026 before 01:00 pm**. Late submissions will incur penalties as per the university policy.
4. **Viva: Attendance at the viva is mandatory**. You must be available to explain your project and answer questions about your implementation. Not attending the viva will result in a **straight zero** for the coursework. Slots will be announced on Blackboard.

Make sure to follow all submission instructions and ensure everything is uploaded on time to avoid any penalties.

Grading Criteria: Your coursework will be graded based on the following areas:

1. Project Setup & Architecture (10 Marks)
2. Database Design & Entity Relationships (15 Marks)
3. API Development (15 Marks)
4. Authentication & Authorization (20 Marks)
5. Business Logic Layer (10 Marks)
6. Unit Testing & Code Quality (10 Marks)
7. Deployment & CI/CD (10 Marks)
8. API Documentation (5 Marks)
9. Version Control & GitHub Usage (5 Marks)

Total Marks: 100

Additional Notes:

- Late submissions without a valid reason may incur penalties, as outlined in the university policy.
- Ensure you follow best practices in coding, including adhering to SOLID principles and maintaining clean, readable code.
- Understanding the context and each requirement is an essential part of your learning process. Be sure to ask questions in advance to prevent any misunderstandings or disappointments.
- Remember that security is a key aspect of your API. Implement appropriate measures, such as password hashing, input validation, and protection against common vulnerabilities like SQL injection and XSS.

Good luck with your project!

.NET Core Backend API Assessment Detailed Rubric (100 Marks)

1. Project Setup & Architecture (10 Marks)

- **Project Structure & Organization (3 Marks)**
    - Proper folder structure (Controllers, Models, Data, Services, etc.) (2)
    - Correct naming conventions and code organization (1)
- **Configuration & Dependency Injection (2 Marks)**
    - Correct setup of appsettings.json for configuration management (1)
    - Proper use of dependency injection (1)
- **CI/CD Pipeline Setup (5 Marks)**
    - Automated build and deployment pipeline configured (GitHub Actions/Azure DevOps) (3)
    - Environment-specific configurations handled securely (2)

---

2. Database Design & Entity Relationships (15 Marks)

- **Database Modeling (5 Marks)**
    - Correct relationships (one-to-many, many-to-many) implemented (3)
    - Proper use of primary & foreign keys (2)
- **Entity Framework Core Usage (5 Marks)**
    - Proper migration setup and execution (2)
    - Usage of Fluent API/Data Annotations for relationships (3)
- **SQLite Database Integration (5 Marks)**
    - Correct setup of SQLite for cross-platform compatibility (3)
    - Ability to seed and retrieve data effectively (2)

---

3. API Development (15 Marks)

- **Controllers and Endpoints (3 Marks)**
    - RESTful principles followed (2)
    - CRUD operations implemented correctly (1)

- **DTOs and Data Validation (3 Marks)**
    - Usage of DTOs for API contracts (2)
    - Proper input validation with Data Annotations (1)
- **Logging Implementation (4 Marks)**
    - Use of ILogger in each entity (2)
    - Logging appropriate levels (info, warning, error) (2)
- **Error Handling (5 Marks)**
    - Proper try-catch blocks and exception handling (3)
    - Global error handling using middleware (2)

---

4. Authentication & Authorization (20 Marks)

- **Identity Framework Integration (5 Marks)**
    - Setup of ASP.NET Core Identity for user management (3)
    - Hashing & salting passwords securely (2)
- **JWT Token Implementation (5 Marks)**
    - Correct generation and validation of JWT tokens (3)
    - Token expiry and refresh mechanisms (2)
- **Authentication Endpoints (5 Marks)**
    - User registration and login API working correctly with email service (3)
    - Role-based access control (2)
- **Security Best Practices (5 Marks)**
    - Use of environment variables for sensitive data (2)
    - Security middleware (CORS, Rate limiting, HTTPS) (3)

---

5. Business Logic Layer (10 Marks)

- **Service Layer Implementation (5 Marks)**
    - Business logic separation from controllers (3)
    - Proper dependency injection of services (2)

- **Repository Pattern Usage (5 Marks)**
    - Abstraction of database access via repositories (3)
    - Proper querying and data fetching methods (2)

---

6. Unit Testing & Code Quality (10 Marks)

- **Unit Tests (5 Marks)**
    - Test cases for API endpoints and services (3)
    - Mocking dependencies correctly (2)
- **Code Readability & Maintainability (5 Marks)**
    - Proper comments and documentation (2)
    - Adherence to SOLID principles (3)

---

7. Deployment & CI/CD (10 Marks)

- **Dockerization (5 Marks)**
    - Correct Dockerfile setup and multi-stage builds (3)
    - Environment variables handled securely (2)
- **Cloud Deployment (5 Marks)**
    - Deployment to cloud platform (Azure/AWS/Heroku) (3)
    - Health checks and logging monitoring (2)

---

8. API Documentation (5 Marks)

- **Swagger Documentation (5 Marks)**
    - Proper API documentation using Swagger (3)
    - Endpoint descriptions and examples provided (2)

---

9. Version Control & GitHub Usage (5 Marks)

- **GitHub Repository Setup (3 Marks)**
    - Proper branching strategy and commit messages (2)

    - README with project instructions (1)

- **Commit History & Best Practices (2 Marks)**
    - Regular commits with meaningful messages (2)

Total Marks: 100

Failure to attend the presentation will result in a potential score of 0% of the total marks, leading to automatic failure. To successfully complete each evaluation, you must attain a minimum score of 40%. Furthermore, an overall minimum of 50% is necessary to pass the module. Therefore, if you achieve a score of 40% in the initial assessment, you must aim for a higher score in the subsequent assessment to ensure your overall marks equal or exceed 50%.